

AEGIS SYSTEM

Smart Patient Triage System

A secure, resilient, and serverless clinical Early Warning System (NEWS2) designed for real-time patient telemetry monitoring. This document serves as a comprehensive guide outlining the system's architecture, its core cloud-native components, chaos validation engineering, local and production deployment processes, and integration protocols for real-world hospital environments.

Document Purpose:	Operations, Technical, and Administrative Guide
Target Audience:	Clinical Leadership, Hospital IT Administrators, DevOps Engineers
System Version:	v1.0.0 (Production Grade)
AWS Deployment Region:	us-east-1 (Northern Virginia)
Release Date:	June 23, 2026
Project Copyright:	© 2026 Rajan Kumar. All rights reserved.
Author's GitHub:	https://github.com/RajGenStack
Author's LinkedIn:	https://www.linkedin.com/in/rajan-kumar42
Author's Instagram:	https://www.instagram.com/rajansxarma?igsh=eDh1bnk1NmVsZicz

1. Executive Summary

In modern acute hospital environments, the early detection of physiological deterioration is critical to saving patient lives. Clinical studies show that delayed intervention is a leading cause of avoidable deaths, cardiac arrests, and unplanned intensive care unit (ICU) admissions. The **AEGIS Smart Patient Triage System** addresses this critical gap.

AEGIS is a serverless, highly resilient clinical decision support system. It ingests continuous vital signs streaming from patient bedside monitors, automatically computes the clinical National Early Warning Score 2 (NEWS2), and displays prioritized triage statuses to medical staff in real time. Because patient monitoring is a zero-downtime operations requirement, AEGIS features a closed-loop, self-healing architecture that detects infrastructure failures and automatically remediates configurations within seconds.

This guide is designed for both medical leadership and IT professionals. It explains *why* each technology was selected, how it benefits clinical safety, and how to operate the system locally and in a secure, production-grade clinical environment.

2. The Clinical Core: The NEWS2 Scoring System

At the heart of the AEGIS system is the **National Early Warning Score 2 (NEWS2)**, an internationally validated clinical tool standard developed by the Royal College of Physicians. NEWS2 evaluates seven key physiological parameters, assigning a score of 0 to 3 for each based on how far they drift from the normal range. The aggregate score determines the triage risk:

- **Low Risk (Aggregate Score 0 - 4):** Standard clinical monitoring. The patient is stable.
- **Medium Risk (Aggregate Score 5 - 6, or any single score of 3):** Requires urgent clinical review. Bedside monitoring must be increased to hourly intervals, and the attending ward nurse must alert the medical team lead.
- **High Risk (Aggregate Score 7 or higher):** Indicates critical clinical danger. Requires immediate emergency medical response and evaluation for intensive care transfer.

Physiological Parameter	3	2	1	0	1	2	3
Respiration Rate (breaths/min)	≤ 8		9 - 11	12 - 20		21 - 24	≥ 25
SpO2 Scale 1 (%) (General)	≤ 91	92 - 93	94 - 95	≥ 96			
SpO2 Scale 2 (%) (COPD target)	≤ 83	84 - 85	86 - 87	88 - 92 ≥ 93 on air	93 - 94 on oxygen	95 - 96 on oxygen	≥ 97 on oxygen
Supplemental Oxygen		Oxygen		Room Air			
Systolic Blood Pressure (mmHg)	≤ 90	91 - 100	101 - 110	111 - 219			≥ 220
Heart Rate (beats/min)	≤ 40		41 - 50	51 - 90	91 - 110	111 - 130	≥ 131
Temperature (°C)	≤ 35.0		35.1-36.0	36.1-38.0	38.1-39.0	≥ 39.1	

Consciousness Level (CVPU)				Alert			Confused/Voice /Pain/Unrespon sive
----------------------------	--	--	--	-------	--	--	------------------------------------

Clinical Note: The system implements SpO2 Scale 2 specifically for COPD (Chronic Obstructive Pulmonary Disease) patients, where normal oxygen levels are expected to sit lower (88% - 92%). Using Scale 1 for these patients could result in over-oxygenation and hypercapnia, creating a clinical hazard. AEGIS eliminates this risk by supporting dynamic scale selection per patient.

3. System Architecture & The 'Why' Behind the Tools

To ensure that this clinical pipeline remains secure, scalable, and highly available, the system is built entirely on top of **Amazon Web Services (AWS)** using serverless infrastructure. Below, we explain each component in non-technical terms, so healthcare operators can understand their role.

3.1 Amazon SQS (Simple Queue Service) — The Shock Absorber

In a busy hospital, hundreds of patient monitors transmit vital signs multiple times per minute. If all these signals hit the database simultaneously, the database can slow down or crash, resulting in lost vitals. Amazon SQS acts as a digital queue (or shock absorber). It temporarily holds all incoming patient payloads in a highly durable buffer. Even if the backend processing slows down temporarily, no data is ever lost; SQS holds the vitals securely until the processing system is ready.

3.2 AWS Lambda — The Serverless Brains

Traditionally, applications run on physical or virtual servers that operate 24/7, costing money even when empty. AWS Lambda is a 'serverless' computing service. Instead of running a continuous server, Lambda runs code only in response to events (e.g., when a patient's vital data lands in the SQS queue). When the queue is empty, the server disappears and costs drop to zero. When a wave of patient vitals comes in, Lambda instantly launches hundreds of parallel execution brains to compute NEWS2 scores and shut down immediately after. This yields significant cost savings and zero maintenance overhead.

3.3 Amazon DynamoDB — The Rapid Record Keeper

Patient data must be recorded instantly. Amazon DynamoDB is a highly scalable, NoSQL database that can perform write operations in single-digit milliseconds. Unlike older database systems that require complex tables and relations, DynamoDB is designed for simple, ultra-fast storage. This ensures that a patient's updated triage score is available on the central nursing station's screen within milliseconds of the bedside monitor sending the signal.

3.4 CloudWatch & EventBridge — The System's Vital Sign Monitors

Just as we monitor patient vitals, AWS CloudWatch monitors the *health of the software pipeline itself*. It tracks metrics like queue backlog, processing latency, and server errors. If a processing error occurs or a database connection slows down, CloudWatch sounds an alarm. Amazon EventBridge catches this alarm and triggers a remediation Lambda (self-healing loop) to automatically diagnose and repair the issue without human intervention, maintaining high clinical uptime.

4. System Resilience: Testing and Self-Healing

In mission-critical clinical software, 'hope' is not a strategy. To guarantee the system's resilience, we implemented a custom **Chaos Engineering Outage Simulator** (outage_simulator.py) that simulates real-world infrastructure failures on the live AWS cloud, combined with a **Self-Healing Loop** that resolves them automatically.

4.1 Chaos Scenarios Tested

- 1. Concurrency Block:** Disables the vital signs processor Lambda by setting its reserved concurrency to zero. This simulates a severe software blockade where no computing brains are permitted to execute.
- 2. Database Configuration Outage:** Modifies the environment configuration of the vital signs processor to point to a non-existent database table. This simulates a broken database mapping or corrupted settings.
- 3. Event Mapping Disable:** Disables the trigger linking SQS to the processor Lambda. This simulates a network trigger break, leaving vital data trapped in the queue.

4.2 The Closed-Loop Self-Healing Process

When an outage is injected, the system heals itself automatically within minutes using the following loop:

Step 1: Failure Detection	CloudWatch Alarms detect the failure (e.g. SQS message backlog increases or processor logs errors).
Step 2: Event Notification	The alarm transitions to ALARM state, sending a state-change event to EventBridge.
Step 3: Auto-Remediation	EventBridge triggers the Remediation Lambda , which inspects the failed resource.
Step 4: Automatic Fix	The Remediation Lambda updates the configuration using the AWS API (e.g., removing the concurrency block or resetting the database table mapping to the valid table).

By testing this loop automatically, we guarantee that the MTTR (Mean Time to Resolution) is under 90 seconds, compared to hours if we had to wait for an on-call IT engineer to respond manually.

5. Hospital Prerequisites & System Integration Guide

Deploying the AEGIS Smart Patient Triage System in a real hospital requires integrating with the existing physical medical devices and hospital networks. Below are the key prerequisites.

5.1 Physical and Middleware Prerequisites

- 1. Bedside Physiological Monitors:** Patient monitors (e.g. Philips, GE, Nihon Kohden) must be configured to stream telemetry data via HL7 v2 (specifically ORU^R01 messages) or FHIR resource updates.
- 2. Integration Middleware:** An integration engine (such as Mirth Connect, AWS IoT Greengrass, or Rhapsody) is required to run on a local hospital server. This middleware intercepts raw bedside monitor signals, converts them to the standardized JSON schema expected by AEGIS, and securely pushes them to the AWS SQS queue over HTTPS.
- 3. Network Gateway:** A secure network gateway configured with outbound proxy rules to permit HTTPS connections only to the designated SQS queue URL in AWS.

5.2 Compliance, Security, and Privacy Requirements

Patient health information is highly sensitive. The production architecture enforces strict compliance with **HIPAA** (Health Insurance Portability and Accountability Act) and **PIPEDA** regulations:

- **Data Encryption in Transit:** All patient telemetry sent over the network is encrypted using TLS 1.3. Direct HTTP connections to Lambda or SQS are blocked.
- **Data Encryption at Rest:** Patient records stored in DynamoDB and messages buffered in SQS are encrypted using customer-managed keys (CMK) stored in **AWS Key Management Service (KMS)**.
- **Private AWS Networking:** Inside AWS, the Lambdas are run inside a private **VPC (Virtual Private Cloud)**. All access to DynamoDB and SQS is routed through **AWS PrivateLink (VPC Endpoints)**. This means patient data never crosses the public internet while inside the cloud.
- **IAM Policies (Least Privilege):** The Processor and API Lambdas do not have full AWS permissions. They are assigned tightly constrained IAM execution roles permitting only `sqs:ReceiveMessage` or `dynamodb:PutItem` on their specific target resources.

6. Local Development and Run Guide

To test or develop the system locally on your computer, follow these step-by-step instructions.

6.1 Prerequisites

- **Node.js:** Version 20 or higher installed.
- **Python:** Version 3.10 or higher installed.
- **AWS CLI:** Installed and configured with permissions to access us-east-1.

6.2 Step-by-Step Local Execution

Step 1: Install Frontend Dependencies

Navigate to the frontend directory and install dependencies:

```
cd frontend  
npm.cmd install
```

Step 2: Configure the API URL

Ensure the API URL in frontend/src/App.jsx points to the live AWS Lambda Function URL:

```
const API_URL = 'https://h2iiorj77xbglciznwm74jad640sqdsd.lambda-url.us-east-1.on.aws/';
```

Step 3: Run the Frontend Development Server

Start the Vite development server locally:

```
npm.cmd run dev
```

Step 4: Launch the Vitals Simulator

Start the Python simulator to push simulated vital signs to the live AWS SQS queue:

```
python vitals_simulator.py --patients 5 --interval 2 --sqs-queue rajgenstack-triage-vitals-queue
```

Once these steps are running, open your web browser to `http://localhost:5173/` to view the live triage dashboard.

7. Production-Grade Deployment Guide

The production deployment enforces an **Infrastructure-as-Code (IaC)** model. No infrastructure should ever be created manually in the AWS Console. Everything is managed through a CI/CD pipeline.

7.1 Deployment Automation (CI/CD)

The pipeline is managed via **GitHub Actions** (.github/workflows/ci-cd.yml) and performs the following automation:

- 1. Quality Gates (Pre-deployment):** Runs Python unit tests to check clinical NEWS2 math logic correctness, validates Terraform formatting (`terraform fmt`), and compiles Vite/React frontend assets to catch syntax errors.
- 2. Terraform Initialization:** Automatically runs `terraform init` inside the `infra/` directory to configure the remote backend.
- 3. Infrastructure Deploy:** Runs `terraform apply -auto-approve` to deploy the updated queue, database, alarms, Lambdas, and EventBridge self-healing rule.

7.2 Production Hardening Checklist

When launching in a production hospital environment, ensure these security standards are enabled:

- **CORS Constraints:** In `infra/modules/lambda`, restrict the API Function URL's CORS configurations to allow only the hospital's official domain name (e.g. `triage.hospital.org`) instead of the development wildcard `*`.
- **WAF Integration:** Attach an **AWS WAF (Web Application Firewall)** to the API Lambda Function URL to prevent DDoS attacks, SQL injection attempts, and restrict traffic only to the hospital's private VPN subnet IP addresses.
- **Backlog Autoscale:** Set SQS queue alarms to automatically provision higher read/write units (RCUs/WCUs) on DynamoDB or scale up Lambda concurrency limits when abnormal patient surges are detected.

This completes the system specifications guide. The AEGIS Smart Patient Triage System provides a critical tool for real-time patient monitoring, ensuring immediate response times to save lives.